

Projektdokumentation · React SPA

4 Routen	4 Views	1 Context	Riot API
----------	---------	-----------	----------

1. Projektbeschreibung

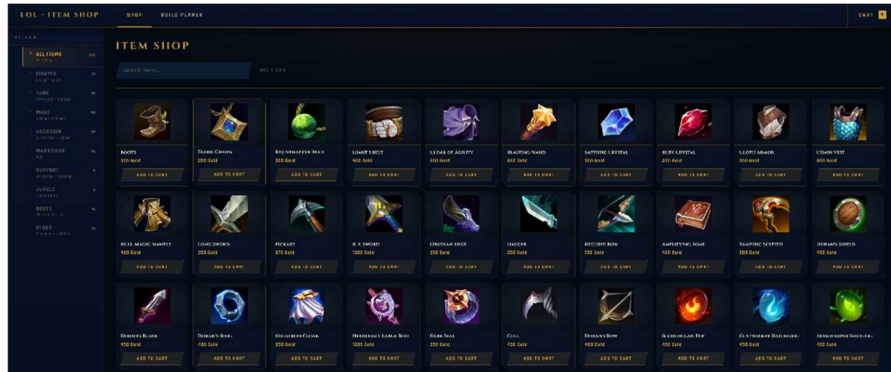
React SPA die alle kaufbaren LoL-Items via Riot Data Dragon API lädt. Filter nach Rollen, Detailansicht, persistenter Warenkorb und 6-Slot Build-Planer mit kombinierter Stat-Anzeige.

Framework	React 18 (Vite)
Routing	react-router-dom v6
State	useState, useEffect, Context API
Styling	Custom CSS – LoL Hextech Design System
Datenquelle	Riot Data Dragon API – öffentlich, kein API-Key nötig
Persistenz	localStorage – Warenkorb bleibt nach Reload erhalten

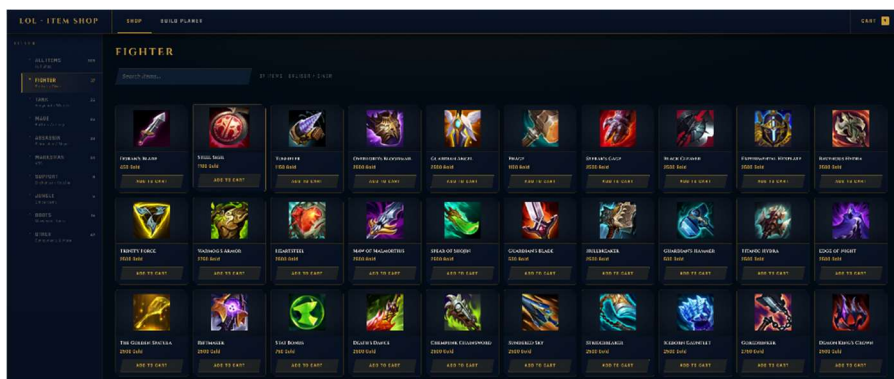
2. Views & Funktionalität

Shop

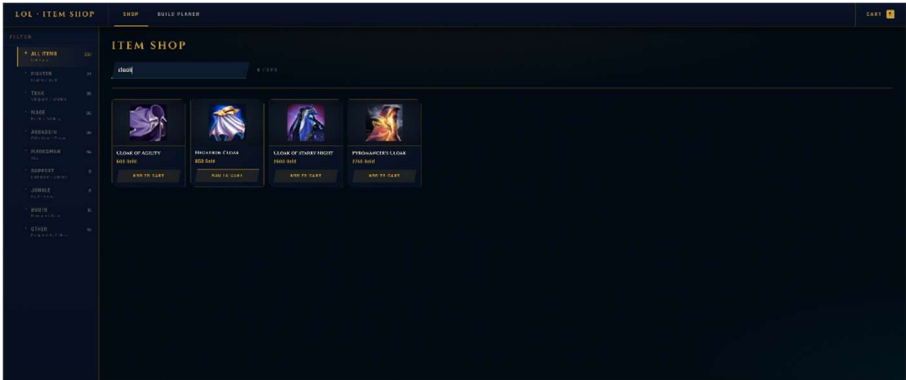
Items ohne Filter



Items mit Filter

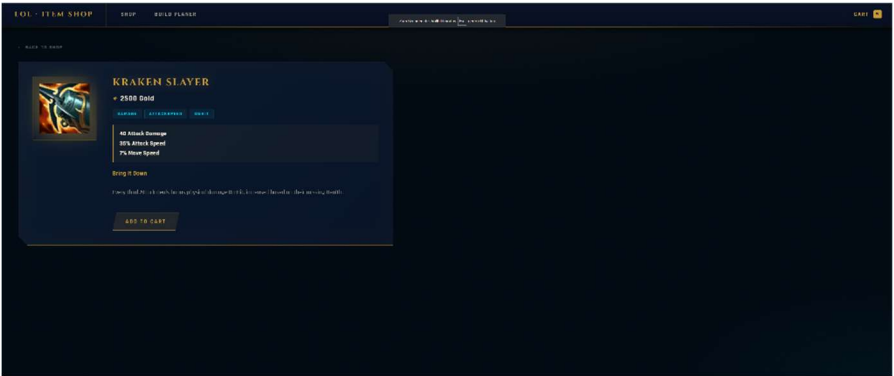


Items im Suchfeld



Suchfeld	Kontrolliertes Input – live-Filter nach Itemname, State = single source of truth
Rollenfilter klicken	Sidebar-Button setzt activeRole, Conditional Styling per ternärem Operator
Filter deaktivieren	Nochmal klicken setzt activeRole auf null – alle Items sichtbar
Item-Bild klicken	react-router-dom Link → /item/:id ohne Seiten-Reload
"Add to Cart"	addToCart() aus CartContext – Cart-Badge in Navbar steigt sofort
Item-Zähler	Jede Kategorie zeigt Anzahl der Items – berechnet via filter().length

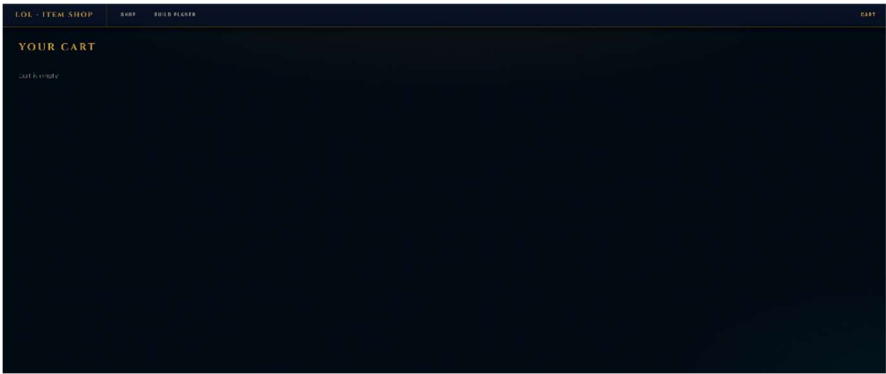
Item Detail



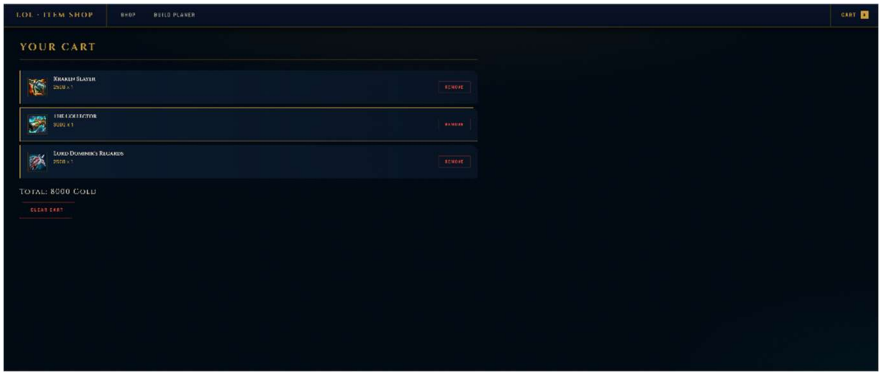
"Add to Cart"	addToCart() aus Context
“← Back”	Link-Komponente navigiert zurück zur Übersicht ohne Reload

Warenkorb

Ohne Items



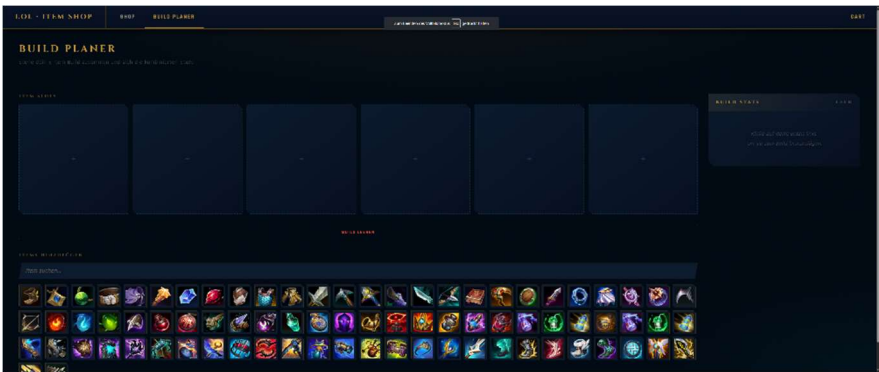
Mit Items



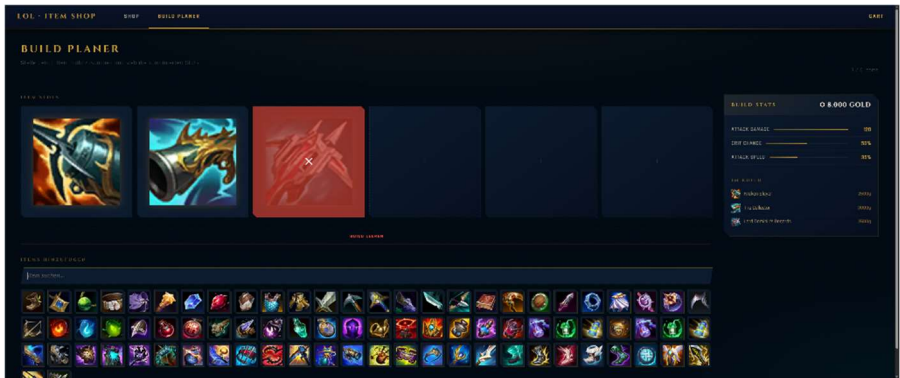
Items im Warenkorb	Name, Bild, Preis $\times$ Menge werden für jeden Eintrag gerendert
Gleiches Item nochmal	<code>addToCart()</code> erhöht quantity via <code>map()</code> – kein Duplikat entsteht
Gesamtpreis	<code>reduce()</code> summiert Preis $\times$ Menge über alle Cart-Items
"Remove" klicken	<code>filter()</code> entfernt Item aus State, Liste re-rendert automatisch
"Clear Cart"	<code>setCart([])</code> – alle Items auf einmal entfernen
Warenkorb leer	Conditional Rendering mit ternärem Operator: 'Cart is empty'
Seite neu laden	localStorage-Persistenz – Warenkorb bleibt vollständig erhalten

Build Planer

Ohne Items

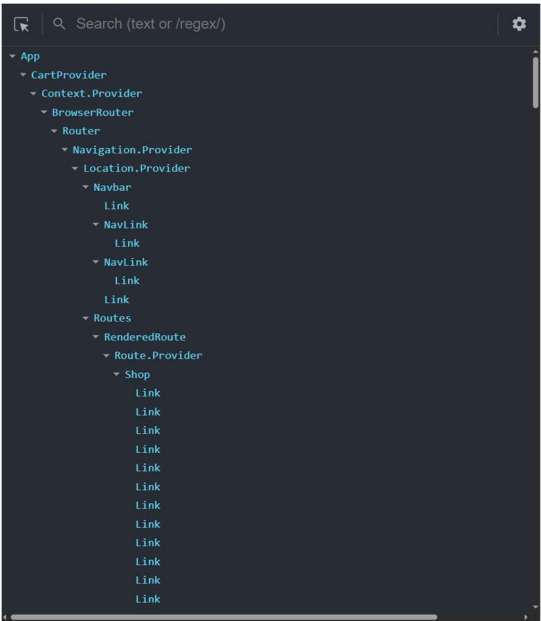


Mit Items



Item-Suche	Kontrolliertes Input filtert Thumbnails live via filter()
Thumbnail klicken	findIndex() findet ersten leeren Slot, Spread-Operator befüllt ihn
Slot hover	Rotes X erscheint – onClick ruft removeFromBuild(idx) auf, Slot wird null
Stats-Panel rechts	reduce() summiert Stats aller Items, Balken = % vom Spielwert-Maximum
Gesamtgold	reduce() berechnet Summe live bei jeder Änderung
Disabled Thumbnails	Items im Build → buildIds (Set), Conditional Styling opacity 0.3
"Build leeren"	setBuild(Array(6).fill(null)) – alle Slots zurücksetzen

3. Component Tree



4. Projektverlauf

Planung vs. Umsetzung

Geplant	Statische Item-Liste mit Dummy-Daten
→ Umgesetzt	Riot Data Dragon API – echte, stets aktuelle Daten ohne API-Key
Geplant	Bootstrap Dark-Theme
→ Umgesetzt	Eigenes Hextech-Design-System (CSS Custom Properties, Clip-Paths)
Geplant	Technische API-Tags als Filterkriterien
→ Umgesetzt	Spielnahe Rollen (Fighter, Tank, Mage...)
Nicht geplant	Build Planer – nachträglich als Kernfeature hinzugefügt

Besonders hartnäckige Fehler

- Unzureichender Item-Filter: Der initiale Filter `item.price > 0` reichte nicht aus, da die Data Dragon API auch interne Komponenten-Items und veraltete Einträge enthält, die technisch einen Preis besitzen. Behoben durch zusätzliche Prüfung auf `purchasable === true` sowie Deduplizierung via `new Map(item.name → item)`.
- Laufzeitfehler beim Routing: Ein Klick auf ein Item führte zu `'Element type is invalid: expected a string but got object'`. Ursache war ein fehlender `default export` in einer Page-Komponente – React importierte ein Modul-Objekt statt eine Funktion. Behoben durch konsequentes `export default FunctionName`; am Ende jeder Page-Datei.
- Rollenfilter mit blinden Flecken: Ein großer Teil der Items erschien in keiner Filterkategorie. Ursache waren zu enge Tag-Kombinationen als Match-Bedingung. Behoben durch eine Auffangkategorie 'Other' für nicht zugeordnete Items.

Ausblick

- Champion-Seite: Riot API liefert empfohlene Items pro Champion – direkt integrierbar
- Preisfilter-Slider und Sortierung nach Preis oder Name
- +/- Buttons im Warenkorb statt nur Remove

5. Anleitung

- `cd lol-shop`
- `npm install`
- `npm start`